

Support Vector Machines — Margins, Support Vectors, and Kernels

The central idea

A Support Vector Machine learns a boundary between classes. A linear SVM balances a wide margin around that boundary against penalties for examples that fall inside the margin or on the wrong side.

When a straight boundary is insufficient, a **kernel** lets the model learn a linear separator in a transformed feature space, which can become a curved boundary in the original space.

We will derive a boundary from a six-point dataset, calculate distances and penalties, then work through linear, polynomial, RBF, and sigmoid kernels. The diagrams use exact geometry or fitted models on explicitly identified synthetic datasets.

1. From feature questions to a geometric boundary

A decision tree predicts by asking a sequence of questions. A linear SVM combines the features into a **score** and predicts from the sign of that score.

With two features:

$$f(x) = w_1x_1 + w_2x_2 + b$$

- $x = (x_1, x_2)$ is one example's feature vector.
- $w = (w_1, w_2)$ is a learned weight vector.
- b is a learned intercept, also called a bias term.
- $f(x)$ is the decision score.

For binary classification, we will encode the target as $y \in \{-1, +1\}$:

$$\hat{y} = \begin{cases} +1 & f(x) > 0 \\ -1 & f(x) < 0 \end{cases}$$

At $f(x) = 0$, the point lies exactly on the boundary; an implementation needs a tie convention.

The labels are codes

-1 and +1 are convenient mathematical labels, not numerical quantities to predict by ordinary regression. They could represent No/Yes, spam/not-spam, or any two classes.

1.1 What is a hyperplane?

The **decision boundary** consists of points satisfying:

$$w^T x + b = 0$$

The superscript T denotes a transpose; $w^T x$ is the **dot product**:

$$w^T x = w_1x_1 + w_2x_2 + \cdots + w_px_p$$

For a linear model, the boundary is:

Number of features	Geometric boundary
1	A point on a number line
2	A straight line

Number of features	Geometric boundary
3	A flat plane
p	A $(p - 1)$ -dimensional hyperplane

The vector w points perpendicular to the boundary, toward increasing scores. The intercept b shifts its position.

With a nonlinear kernel, the hyperplane is in the transformed feature space. Its image in the original input coordinates need not be a straight line or flat plane.

2. Our running dataset

Consider six examples with two numerical features, already expressed on comparable scales:

Point	x_1	x_2	Class y
A	1	1	-1
B	1	3	-1
C	2	2	-1
D	4	2	+1
E	5	1	+1
F	5	3	+1

One row is a sample. The two coordinates are features, and the class is the known training target.

The negative examples lie to the left and the positive examples to the right. Many vertical lines separate them perfectly:

```
x1 = 2.4 separates all six points.
x1 = 3.0 separates all six points.
x1 = 3.8 separates all six points.
```

All three have 100% training accuracy. Which boundary should we prefer?

3. Margin: how much clearance does the boundary leave?

The **geometric margin of a correctly separating boundary** is its smallest perpendicular distance to a training point.

For $x_1 = 2.4$:

- The nearest negative point, C, is $2.4 - 2 = 0.4$ away.
- The nearest positive point, D, is $4 - 2.4 = 1.6$ away.
- The smaller clearance is 0.4.

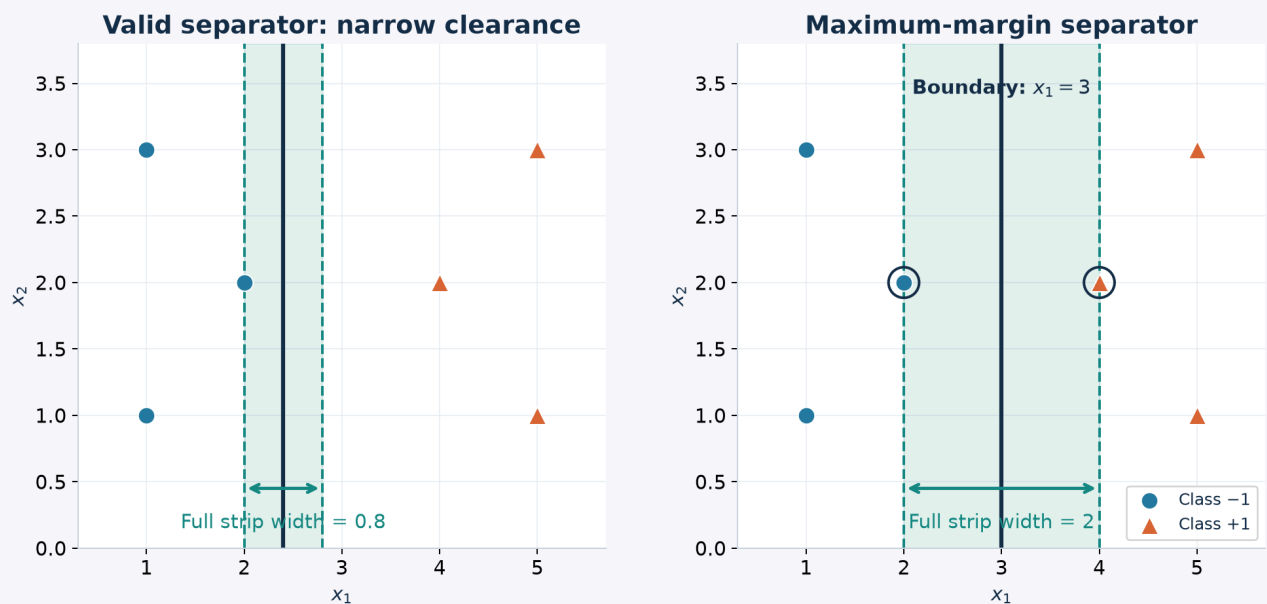
For $x_1 = 3$:

- C is 1 unit away.
- D is 1 unit away.
- The smaller clearance is 1.

The second boundary leaves more room before a nearby point crosses it.

01 | A boundary is not enough

Both lines classify the six points correctly. The closest-point clearance is different.



Dashed lines mark equal clearance on both sides. Rings identify the two support vectors in the right panel.

One-sided margin versus full margin width

In these notes, **one-sided margin** means the distance from the decision boundary to a margin edge. **Full margin width** means the distance between the two parallel margin edges.

For our maximum-margin boundary, the one-sided margin is 1 and the full width is 2. Both conventions appear in textbooks, so check which distance a formula describes.

3.1 Why is $x_1 = 3$ optimal here?

C and D are only 2 units apart. Any line separating them must pass between them. The sum of their perpendicular distances to that line cannot exceed their Euclidean separation of 2.

Therefore, their smaller distance cannot exceed 1. The line $x_1 = 3$ achieves 1 for both, and all other samples lie farther away. No other separator can give a larger minimum clearance for this dataset.

3.2 Why might a larger margin help?

A wider margin can make predictions less sensitive to small changes in input values. This motivates margin maximization as a way to control model complexity.

It does not guarantee correctness on unseen data. Generalization also depends on the data distribution, noise, feature representation, scaling, and regularization.

4. Derive w , b , and the margin

The boundary is $x_1 = 3$, so one score representing it is:

$$f(x) = x_1 - 3$$

Therefore:

$$w = (1, 0), \quad b = -3$$

The second feature receives weight zero in this example because horizontal position alone provides the widest separation. This is a result of this dataset, not a general property of SVMs.

4.1 Why do the margin edges have scores -1 and +1?

The equations $x_1 - 3 = 0$ and $10x_1 - 30 = 0$ describe the same boundary. Scaling both w and b by a positive constant changes scores without changing classifications or distances.

For a hard-margin SVM, we remove that scale ambiguity by requiring:

$$y_i(w^T x_i + b) \geq 1$$

The nearest points can then lie on:

$$w^T x + b = -1 \quad \text{and} \quad w^T x + b = +1$$

In our example, these lines are $x_1 = 2$ and $x_1 = 4$.

The scores -1 and +1 are a normalization used in the optimization. They are not probabilities and are not automatically distances of one unit for every SVM.

4.2 Convert score to perpendicular distance

For a linear boundary, the signed perpendicular distance is:

$$d(x) = \frac{w^T x + b}{\|w\|}$$

The weight norm is:

$$\|w\| = \sqrt{w_1^2 + w_2^2 + \dots + w_p^2}$$

Here:

$$\|w\| = \sqrt{1^2 + 0^2} = 1$$

At the normalized margin edges, the score magnitude is 1, giving:

$$\text{One-sided margin} = \frac{1}{\|w\|}$$

$$\text{Full margin width} = \frac{2}{\|w\|}$$

Our full width is $2/1 = 2$.

4.3 Verify every training row

Point	True label y	Score $f(x) = x_1 - 3$	$yf(x)$	Location
A	-1	-2	2	Beyond negative margin edge
B	-1	-2	2	Beyond negative margin edge
C	-1	-1	1	On negative margin edge
D	+1	+1	1	On positive margin edge
E	+1	+2	2	Beyond positive margin edge
F	+1	+2	2	Beyond positive margin edge

The product $yf(x)$ is called the **signed functional margin**. It is positive when the score agrees with the true label and negative when it disagrees.

4.4 Predict a new point

For $x = (3.6, 1.5)$:

$$f(x) = 1(3.6) + 0(1.5) - 3 = 0.6$$

The score is positive, so predict +1. Its perpendicular distance from the boundary is $0.6/1 = 0.6$.

The point is inside the margin strip but still on the positive side. **The margin is not a third prediction class.** The model predicts a class even for points inside it.

5. Support vectors: which training points anchor the solution?

In our separable example, C and D touch the margin edges. They are the **support vectors**.

Moving a distant point slightly without bringing it closer to the margin need not change this solution. Moving C or D inward can change the available gap and the learned boundary.

For a standard kernel SVM, support vectors are training samples with nonzero coefficients in the fitted decision function. This definition remains useful when data overlaps and the simple “closest points on the edges” picture is incomplete.

🔗 Soft-margin support vectors need not sit on an edge

They may lie on a margin edge, inside the margin, or on the wrong side of the decision boundary. A misclassified outlier can be a support vector too.

A model can have many support vectors. The term does not promise that only a tiny number of rows matter.

All training rows participate in finding the solution. Support vectors are identified by fitting, not chosen beforehand as the points that look close to an arbitrary line.

6. Hard-margin SVM: the optimization objective

A **hard-margin SVM** requires every training point to be correctly classified and outside or on its margin edge:

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

subject to:

$$y_i(w^T x_i + b) \geq 1 \quad \text{for every training point } i$$

Why minimize the squared norm? The margin width is $2/\|w\|$, so a smaller norm gives a wider margin under the normalization above. Squaring preserves the minimizer and is convenient for optimization; the factor $1/2$ simplifies derivatives.

This is a constrained convex optimization problem for the standard linear formulation. A solver determines w and b together rather than enumerating every possible line by hand.

6.1 When is hard margin unsuitable?

If opposite labels overlap so that no linear separator exists, these constraints cannot all be satisfied. Even if separation is possible, an unusual observation can force a narrow margin.

Soft margin relaxes the constraints while charging a penalty for violations.

7. Soft margin: a correct point can still violate the margin

Introduce one nonnegative **slack variable**, ξ_i , per training sample:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

For a fixed score, the smallest required slack is:

$$\xi_i = \max(0, 1 - y_i f(x_i))$$

This is also the **hinge loss**.

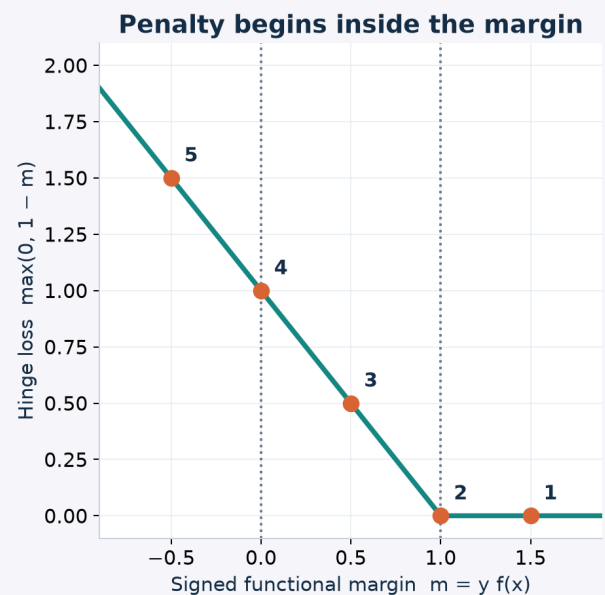
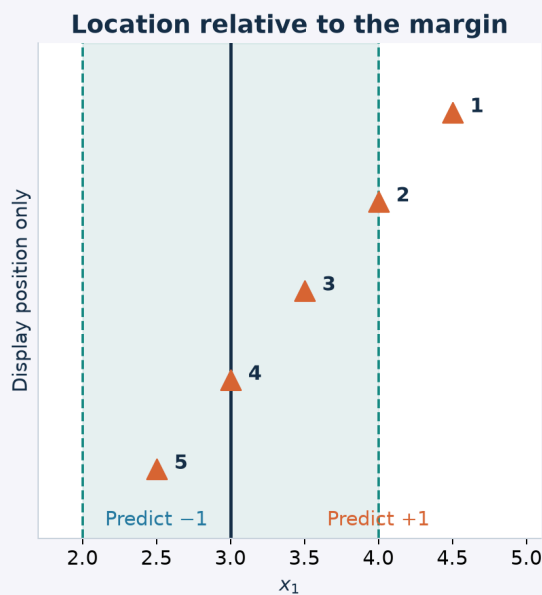
7.1 Five positions, five penalties

Keep the score $f(x) = x_1 - 3$. Consider five hypothetical points whose true class is +1:

Point	x_1	$yf(x)$	Hinge loss	Interpretation
1	4.5	1.5	0	Correct and beyond the margin
2	4.0	1.0	0	Correct and on the margin
3	3.5	0.5	0.5	Correct but inside the margin
4	3.0	0.0	1.0	Exactly on the decision boundary
5	2.5	-0.5	1.5	Misclassified

02 | Correct predictions can still incur a penalty

Fixed example score: $f(x) = x_1 - 3$. Every numbered point below has true label +1.



Point 3 is correctly classified but penalized. Point 4 is on the boundary. Point 5 is misclassified.

The diagram evaluates a fixed boundary. It does not claim that this boundary would remain optimal if these hypothetical points were added and the model retrained.

🔗 Margin violation is broader than misclassification

A point with $0 < yf(x) < 1$ is classified correctly but incurs a positive penalty. SVM training cares about clearance as well as the sign of the prediction.

For $y = -1$, the same formula works. If the score is -0.5 , then $yf(x) = (-1)(-0.5) = 0.5$: correct, but inside the negative-side margin.

8. C: balance margin width against violations

The standard soft-margin objective is:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w^T x_i + b))$$

There are two competing parts:

Part	What it encourages
$\frac{1}{2} \ w\ ^2$	Smaller weights and a wider normalized margin
$C \sum_i \text{hinge loss}_i$	Fewer or smaller training margin violations

Small C: violations receive less weight relative to the norm penalty. This is stronger regularization and can underfit.

Large C: violations receive more weight. The model works harder to reduce them, potentially fitting noise or outliers.

8.1 Compare two candidate solutions

Suppose two candidates have these values on the same training data:

Candidate	Norm penalty	Total hinge loss
A	0.5	2.0
B	2.0	0.2

At $C = 0.1$:

$$J_A = 0.5 + 0.1(2) = 0.7$$

$$J_B = 2 + 0.1(0.2) = 2.02$$

Candidate A has the smaller objective.

At $C = 10$:

$$J_A = 0.5 + 10(2) = 20.5$$

$$J_B = 2 + 10(0.2) = 4$$

Candidate B now wins. These are illustrative candidate scores; a real solver searches across the feasible parameter space.

8.2 What C does not mean

- It is not the allowed number or percentage of wrong predictions.
- It does not penalize only misclassifications; correctly classified points inside the margin also contribute.
- A finite large C does not automatically turn inseparable data into a hard-margin problem.
- Changing C does not guarantee a particular number of support vectors or training errors.

The scale of C depends on feature scaling, sample size, sample weights, and whether the formulation sums or averages its loss. Compare values within a consistent pipeline and implementation.

9. Feature scaling changes the geometry

Suppose one feature is Age, ranging from 20 to 60, while another is annual Income, ranging from 200,000 to 2,000,000.

Without scaling, a difference of 100,000 in Income can dominate a difference of 5 in Age when computing distances. Scaling also changes how weight magnitudes are penalized in a linear SVM.

A common numerical transformation is standardization:

$$x'_j = \frac{x_j - \mu_j}{s_j}$$

Here, μ_j and s_j are the training-set mean and standard deviation of feature j .

Standardization gives features comparable numerical scales. It does not make them equally useful; the model still learns which patterns matter.

⚠ Fit preprocessing only on training data

Compute scaling and imputation parameters using the training portion, then reuse them on validation, test, and future examples. In cross-validation, fit preprocessing separately inside each training fold.

A pipeline keeps these operations together. Sparse text features often need sparse-compatible scaling or normalization rather than centering that would destroy sparsity.

10. When a straight boundary cannot work

Now switch to a second, deliberately different dataset:

Point	x_1	x_2	Class
P	-1	-1	+1
Q	-1	+1	-1
R	+1	-1	-1
S	+1	+1	+1

Equal signs have label +1; opposite signs have label -1. This is an **XOR-style** arrangement.

The two classes occupy opposite diagonals. No single straight line can separate them. Increasing C cannot give a linear model the missing nonlinear shape.

10.1 Add an interaction feature

Define:

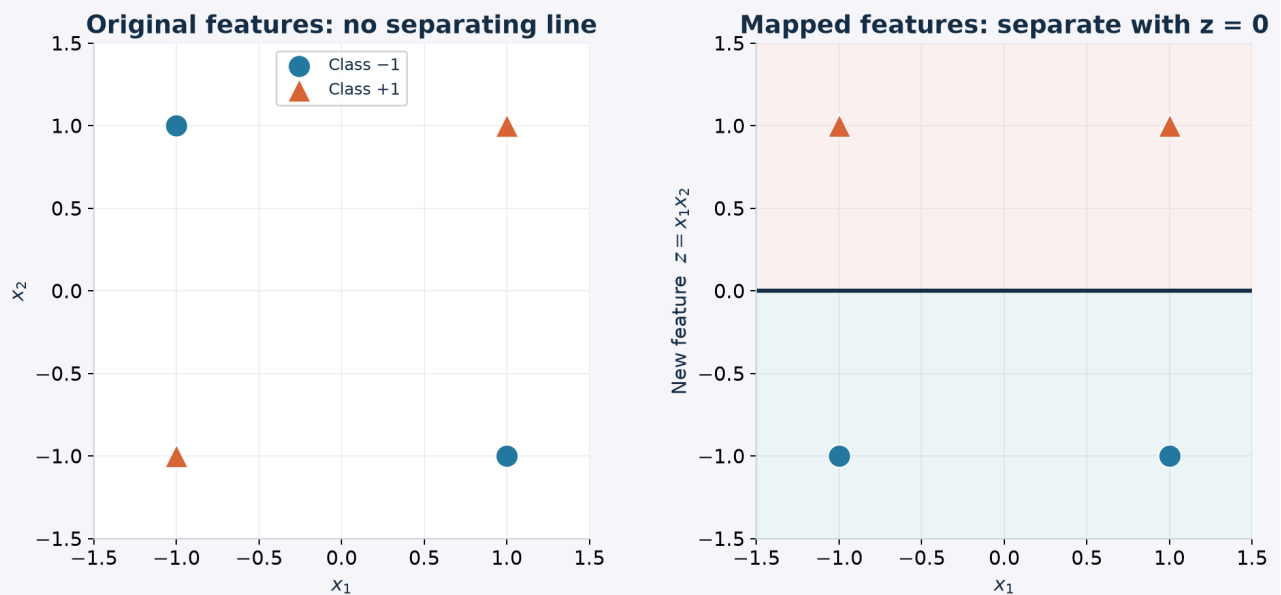
$$z = x_1 x_2$$

Point	New feature z	Class
P	+1	+1
Q	-1	-1
R	-1	-1
S	+1	+1

The rule "predict from the sign of z " separates the classes perfectly. It is linear in the new feature z , but nonlinear in the original pair (x_1, x_2) .

03 | A nonlinear pattern becomes a linear separation

XOR-style labels: equal signs belong to +1, opposite signs belong to -1.



This explicit map illustrates the idea. A kernel computes mapped dot products without constructing every mapped coordinate.

A **feature map** is a transformation $\phi(x)$ that constructs a new representation of an input. It can introduce squared terms, products, or other functions.

This explicit example shows why changing representation helps. The kernel trick allows a related computation without explicitly constructing all mapped coordinates.

11. The kernel trick: replace mapped dot products

A kernel function satisfies:

$$K(x, z) = \phi(x)^T \phi(z)$$

Here, x and z are two input examples. In this equation, z names a second example, not the single interaction feature from the previous section.

The kernel directly returns the dot product of the transformed examples. An SVM can be written in terms of these pairwise dot products, allowing the substitution of K without explicitly building the full feature map.

🔗 A kernel does not itself output a class

It compares two input vectors under a chosen representation. The SVM learns coefficients and combines many such comparisons to produce a decision score.

Not every similarity function is a valid positive-semidefinite kernel. Validity matters for the usual convex SVM optimization guarantees.

11.1 Where support vectors appear in prediction

A fitted binary kernel SVM has the decision function:

$$f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b$$

- SV is the set of support vectors.
- x_i is a stored support-vector input.

- y_i is its training label.
- α_i is a nonnegative learned coefficient.
- b is the learned intercept.

For standard soft-margin C-SVC without extra sample weighting, the dual coefficients satisfy $0 \leq \alpha_i \leq C$. Non-support vectors have coefficient zero.

Prediction compares the new input with support vectors, weights the kernel values, adds them, and applies the sign rule.

With a linear kernel, the weighted sum can be collected into a single vector w , so prediction can use $w^T x + b$ directly.

11.2 Does the kernel trick make everything cheap?

It can avoid creating a very large or infinite-dimensional feature representation. However, exact kernel training can require many pairwise kernel evaluations and substantial memory. Avoiding explicit features does not guarantee low total computational cost.

12. Linear kernel

$$K(x, z) = x^T z$$

The feature map is the original representation: $\phi(x) = x$.

For:

$$x = (1, 2), \quad z = (3, 4)$$

the kernel value is:

$$K(x, z) = 1(3) + 2(4) = 11$$

A dot product is influenced by both direction and magnitude. It is not the same as Euclidean distance or cosine similarity unless appropriate normalization is applied.

What a linear kernel can express

The boundary is linear in the supplied features. If those features already contain interactions, text representations, or other transformations, “linear in the supplied features” need not mean simple in the original raw inputs.

Linear models are useful baselines and can suit high-dimensional sparse representations. They do not require perfect linear separability because soft margin allows violations.

13. Polynomial kernel

The common polynomial kernel is:

$$K(x, z) = (\gamma x^T z + r)^d$$

- d is the degree.
- γ scales the dot product.
- r is an offset, called `coef0` in scikit-learn.

With $\gamma = 1$, $r = 1$, $d = 2$, and the vectors above:

$$K(x, z) = (11 + 1)^2 = 144$$

13.1 See the feature map in the algebra

For two-dimensional inputs, a homogeneous degree-2 kernel has $r = 0$ and $\gamma = 1$:

$$K(x, z) = (x_1 z_1 + x_2 z_2)^2$$

Expanding:

$$K(x, z) = x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2$$

This is the dot product of:

$$\phi(x) = (x_1^2, \sqrt{2}x_1 x_2, x_2^2)$$

and the corresponding $\phi(z)$.

The product $x_1 x_2$ from the XOR example is present automatically. That is why a degree-2 polynomial representation can express this interaction.

For $x = (1, 2)$ and $z = (3, 4)$:

$$\begin{aligned}\phi(x) &= (1, 2\sqrt{2}, 4), & \phi(z) &= (9, 12\sqrt{2}, 16) \\ \phi(x)^T \phi(z) &= 9 + 48 + 64 = 121 = (x^T z)^2\end{aligned}$$

13.2 Degree and offset

With zero offset, the expansion consists of terms of exactly degree d . With a positive offset, lower-degree terms are also included.

Increasing degree allows more complex interactions, but can also increase sensitivity to scaling and overfitting. A polynomial kernel does not mean the boundary must look like one familiar curve such as a parabola; its shape depends on the learned coefficients and input dimension.

14. RBF kernel: distance-based similarity

The **radial basis function** or **Gaussian** kernel is:

$$K(x, z) = \exp(-\gamma \|x - z\|^2), \quad \gamma > 0$$

It depends only on distance between inputs:

- Identical inputs give $K(x, x) = 1$.
- Nearby inputs have values close to 1.
- Distant inputs have values close to 0.

The value is a kernel similarity, not a class probability.

14.1 Calculate a kernel value

For $x = (1, 2)$ and $z = (3, 4)$:

$$\|x - z\|^2 = (1 - 3)^2 + (2 - 4)^2 = 8$$

At $\gamma = 0.5$:

$$K(x, z) = e^{-0.5(8)} = e^{-4} \approx 0.0183$$

At $\gamma = 0.1$:

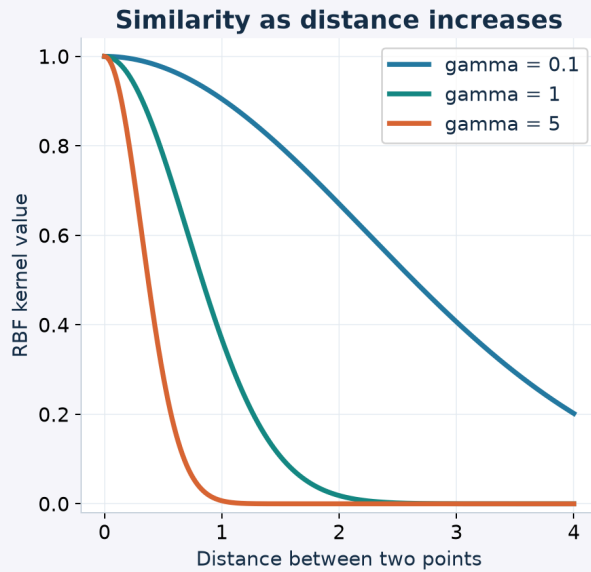
$$K(x, z) = e^{-0.8} \approx 0.4493$$

The same pair becomes much more similar under the smaller gamma.

14.2 Gamma controls the distance scale

05 | Gamma changes the reach of RBF similarity

$K(x, z) = \exp(-\text{gamma} \times \text{distance}^2)$. Larger gamma makes similarity fall more quickly.



Small gamma

Broad similarity

Distant points remain related

Large gamma

Local similarity

Only nearby points remain similar

Not a probability

Kernel values compare inputs.

The learned SVM still combines them.

Distance is measured after preprocessing. Changing feature scales changes the kernel values.

Small gamma: similarity falls slowly with distance. The model compares points over a broader neighborhood.

Large gamma: similarity falls rapidly. Contributions are more local, and the model can form narrow regions around observations.

Another common parameterization uses a width σ :

$$\gamma = \frac{1}{2\sigma^2}$$

Large width means small gamma. These are inverse descriptions of the same distance scale.

The RBF kernel corresponds to an infinite-dimensional feature space. No infinite array needs to be constructed: each kernel evaluation is a finite calculation using the original inputs.

14.3 A small prediction calculation

Suppose an illustrative fitted score has two support-vector contributions:

$$f(x) = 0.8K(s_+, x) - 0.8K(s_-, x) + 0.1$$

Assume the evaluated kernel values for a new input are:

$$K(s_+, x) = 0.9, \quad K(s_-, x) = 0.2$$

Then:

$$f(x) = 0.8(0.9) - 0.8(0.2) + 0.1 = 0.66$$

The sign is positive, so predict +1. The result is not "66% probability." The coefficients and kernel values are an illustrative score calculation, separate from our six-point linear fit.

15. Sigmoid kernel

The sigmoid kernel has the form:

$$K(x, z) = \tanh(\gamma x^T z + r)$$

For the same vectors, $x^T z = 11$. With $\gamma = 0.1$ and $r = 0$:

$$K(x, z) = \tanh(1.1) \approx 0.8005$$

The hyperbolic tangent maps inputs to values between -1 and $+1$. This resembles a neural-network activation, but choosing it as an SVM kernel does not train a multilayer neural network.

Unlike the linear, usual nonnegative-parameter polynomial, and positive-gamma RBF kernels, the sigmoid form is **not positive semidefinite for arbitrary parameters and input domains**. Its behavior therefore requires additional care; the standard convex-kernel interpretation is not automatically valid for every sigmoid configuration. [LIBSVM kernel guidance](#)

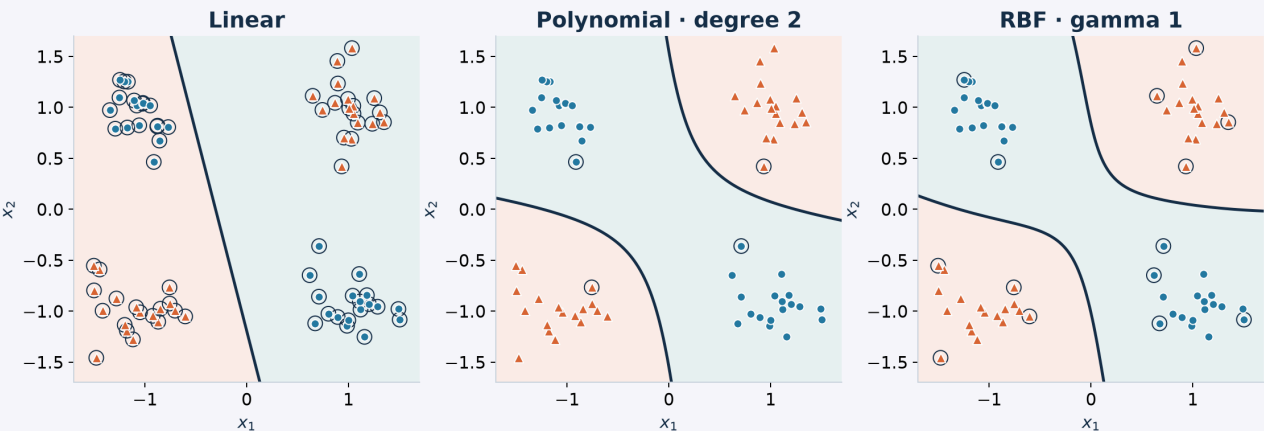
Also distinguish it from logistic regression: logistic regression uses the logistic sigmoid to map a score to a probability. This kernel uses `tanh` to compare two inputs and does not output a class probability.

16. Compare the basic kernels

Kernel	Formula	Main idea	Main controls
Linear	$x^T z$	Linear boundary in supplied features	C
Polynomial	$(\gamma x^T z + r)^d$	Polynomial feature combinations	C, degree, gamma, offset
RBF	$e^{-\gamma \ x - z\ ^2}$	Local distance-based comparisons	C, gamma
Sigmoid	$\tanh(\gamma x^T z + r)$	Saturating dot-product comparison	C, gamma, offset; validity caveats

04 | Same points, different kernels

Actual SVC fits on one fixed synthetic dataset. Colored regions show the predicted class.



Rings show fitted support vectors. These are illustrative training boundaries, not a ranking of generalization performance.

This diagram uses noisy clusters around the four XOR corners. All three panels use the same points. The linear model cannot express the diagonal class structure well; polynomial and RBF models can represent suitable nonlinear boundaries.

These are training illustrations, not evidence that one kernel always generalizes better. The displayed parameters are part of each example, and validation is still needed.

17. Tune C and gamma together

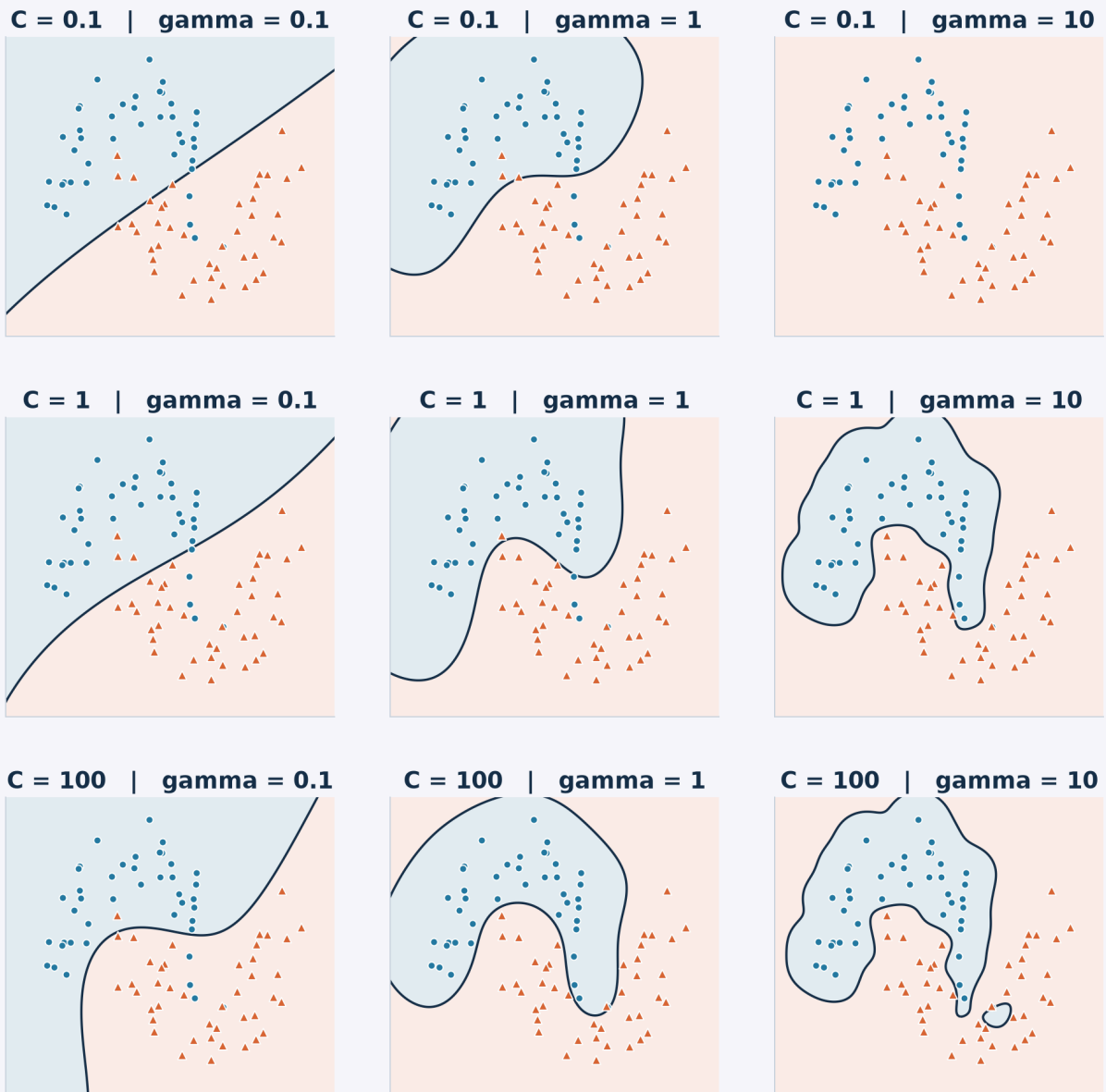
C and gamma influence different parts of an RBF model:

Parameter	Role
C	Relative penalty for training margin violations
Gamma	Distance scale of the kernel representation

Small gamma can produce a representation too broad to capture local distinctions. Large C cannot necessarily repair that. Large gamma can make highly localized regions possible; large C may encourage fitting difficult training examples within that representation.

06 | C and gamma act together

Same standardized training data in every panel. Only the two hyperparameters change.



Higher complexity is not proof of better prediction. Select C and gamma with held-out or cross-validation performance.

The same standardized two-moons training dataset appears in every panel. Only C and gamma change. A boundary that wraps tightly around training points is not automatically better on new points.

A practical search compares combinations on logarithmic scales, for example:

```
C:      0.1, 1, 10, 100
gamma: 0.01, 0.1, 1, 10
```

These are example search ranges, not universal settings. The appropriate range depends on the representation and scaling. Use cross-validation and inspect performance near the best region rather than choosing a boundary because it looks attractive.

18. Multiclass classification, probabilities, and regression

18.1 More than two classes

The basic formulation separates two classes. Common extensions combine binary classifiers:

- **One versus rest:** train one classifier per class against all other classes.
- **One versus one:** train a classifier for every pair of classes.

For three classes A, B, and C, one-versus-one builds A/B, A/C, and B/C models. With k classes, there are $k(k-1)/2$ pairs.

Scikit-learn's `SVC` fits pairwise models internally. Its default one-versus-rest-shaped decision output does not change that internal training strategy. `LinearSVC` normally uses one-versus-rest. [SVM multiclass documentation](#)

18.2 Scores are not probabilities

An SVM decision score measures which side of a learned boundary an example falls on. It is not inherently calibrated as a probability, and scores from separately fitted models need not be comparable.

Scikit-learn's `SVC(probability=True)` enables an additional probability-estimation procedure with extra fitting cost. Probability-based decisions can differ from the classifier's direct decision rule. [SVC documentation](#)

18.3 Support Vector Regression

SVR adapts the method to numeric targets. Instead of separating classes, it fits a function with an **epsilon-insensitive tube** around it.

The loss is:

$$L_{\epsilon}(y, f(x)) = \max(0, |y - f(x)| - \epsilon)$$

Suppose $\epsilon = 2$ and the model predicts 50:

Actual value	Absolute error	Epsilon-insensitive loss
51	1	0
52	2	0
55	5	3

Errors inside the tolerance tube incur no loss. Errors outside are penalized, balanced against model complexity using C . The epsilon tube is a loss tolerance, not a statistical confidence interval.

SVR can also use linear or nonlinear kernels. Its extrapolation behavior depends on the kernel; an RBF model should not be assumed to continue a linear trend beyond the observed data.

19. Practical settings and model choice

Setting	Meaning
<code>C</code>	Relative penalty for margin violations
<code>kernel</code>	Linear, polynomial, RBF, sigmoid, or another supported kernel
<code>gamma</code>	Kernel scale for RBF, polynomial, and sigmoid
<code>degree</code>	Polynomial degree; irrelevant to RBF and linear kernels
<code>coef0</code>	Offset in polynomial and sigmoid kernels
<code>class_weight</code>	Adjusts the effective penalty for examples of different classes
<code>probability</code>	Enables additional probability estimation in SVC

Scikit-learn's `gamma="scale"` is data-dependent; it is not a fixed universal constant. Preprocessing can therefore affect both distances and the resulting default gamma. [SVC parameters](#)

`SVC(kernel="linear")` and `LinearSVC` are related but not identical estimators. They differ in solver, default loss, and other details. Linear solvers can be more suitable when the number of rows makes exact kernel training expensive. [SVM guide](#)

A complete evaluation sequence

1. Define the target and features available at prediction time.
2. Create a suitable held-out test set, respecting time or group structure when relevant.
3. Put imputation, encoding, scaling, and the estimator in a training pipeline.
4. Compare candidate kernels and hyperparameters using cross-validation on the training portion.
5. Select using the task's metric, such as recall, F1, or regression error.
6. Refit the chosen pipeline on the training data.
7. Evaluate once on the held-out test set for the final assessment.

A high accuracy can conceal failure on a rare class. Class weights and a suitable metric can help, but must be evaluated rather than assumed to solve imbalance.

20. Strengths and limitations

Strength	Important limit
Explicit margin-based regularization	Generalization still depends on tuning and data quality
Linear methods can work well with many features	High dimensionality does not make SVM immune to irrelevant features or overfitting
Kernels allow nonlinear boundaries	Exact kernel training can become expensive as sample count grows
Support-vector representation can be compact	Many training rows may become support vectors, increasing prediction cost
A linear score can be inspected through weights	Nonlinear kernel models are harder to summarize as simple rules

SVMs are sensitive to feature geometry. RBF SVMs explicitly depend on distances, so they are not exempt from difficulties caused by poorly scaled or uninformative high-dimensional features.

A linear model can predict using one dot product. A kernel model usually evaluates the kernel against its support vectors, so prediction cost depends on their number and on the cost of the kernel.

21. Common misconceptions

Misconception	More accurate statement
"Any perfectly separating line is equally good."	Different lines can have different minimum clearances.
"Margin always means $2/\ w\ $."	That is the full width under canonical normalization; one-sided margin is $1/\ w\ $.
"Support vectors are always correctly classified edge points."	Soft-margin support vectors can lie inside the margin or be misclassified.
"C penalizes only wrong labels."	It penalizes hinge loss, including correct points inside the margin.
"Large C fixes a nonlinear pattern."	It changes the tradeoff, not the feature representation of a linear model.
"An RBF value is a probability."	It is a kernel similarity between two inputs.
"The kernel explicitly builds all higher-dimensional coordinates."	The trick computes mapped dot products without constructing them.
"Kernel methods are always faster than explicit features."	Pairwise kernel calculations can still be expensive.
"A sigmoid kernel is logistic regression."	They use different functions for different purposes.

22. Self-check

🕒 For the six-point dataset, what are the boundary, weights, and intercept? >

The boundary is $x_1 = 3$, with $w = (1, 0)$ and $b = -3$ in the normalized representation.

🕒 Which points are the support vectors in that example? >

C at (2, 2) and D at (4, 2). They touch the two margin edges.

🕒 What are the one-sided margin and full width? >

They are 1 and 2, respectively, because $\|w\| = 1$.

🕒 If a true +1 point has score 0.3, is it correct and does it incur hinge loss? >

It is correctly classified, but its hinge loss is $1 - 0.3 = 0.7$ because it lies inside the margin.

🕒 If a true -1 point has score +0.4, what is its hinge loss? >

Its functional margin is $(-1)(0.4) = -0.4$, so its hinge loss is 1.4. It is misclassified.

🕒 Why can we not measure geometric distance directly from every SVM score? >

A linear score must be divided by $\|w\|$. Kernel scores refer to a transformed representation and are not generally distances in the original coordinates.

🕒 What does lowering C do? >

It reduces the relative penalty on training margin violations, strengthening regularization in the stated objective.

🕒 Which feature separates the XOR-style example? >

The interaction x_1x_2 . Its sign matches the labels in the example.

🕒 What must a kernel compute for the feature-space interpretation? >

A mapped dot product: $K(x, z) = \phi(x)^T \phi(z)$.

🕒 What happens to RBF similarity when gamma increases at a fixed nonzero distance? >

It decreases. Similarity becomes more local.

🕒 Why tune C and gamma together? >

C sets the penalty tradeoff while gamma sets the representation's distance scale. Their effects interact.

❓ Does a score of 0.8 mean 80% probability? >

No. Probability estimation requires an additional procedure or calibration.

❓ Does every sigmoid parameter choice define a valid positive-semidefinite kernel? >

No. The sigmoid kernel requires care because validity is not guaranteed for arbitrary parameters and inputs.

References

- [Scikit-learn: Support Vector Machines](#)
- [Scikit-learn: SVC parameters and behavior](#)
- [Scikit-learn: RBF parameter effects](#)
- [LIBSVM: kernel guidance and sigmoid caveats](#)

Next: [02 - Support Vector Machines - Practical Implementation](#)